

WaitWatcher & WaitGuard

Detecting Cache Side Channels through Spurious Wakeups

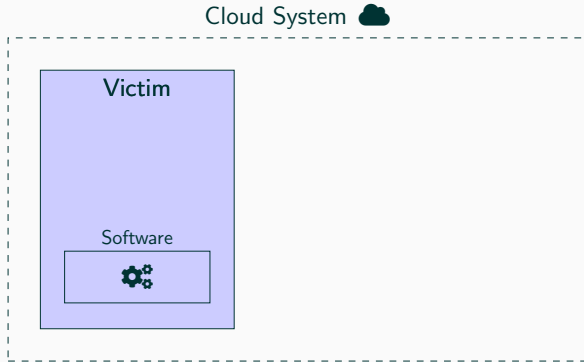
Lukas Lamster

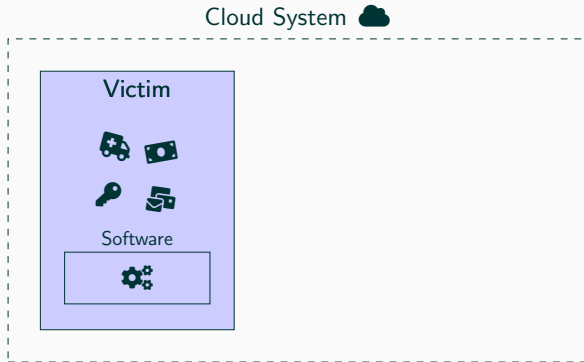
September 22, 2025

ISEC – Graz University of Technology

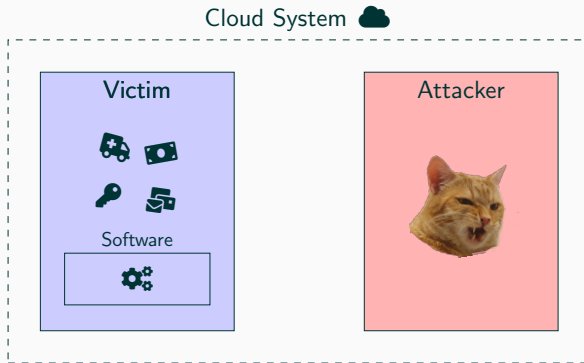
- 💡 Motivation and Background
- 🔍 Detecting Flushes
- ⚙️ The WaitWatcher Framework
- 🛡️ WaitGuard and Evaluation
- 🚩 Conclusion

Background and Motivation

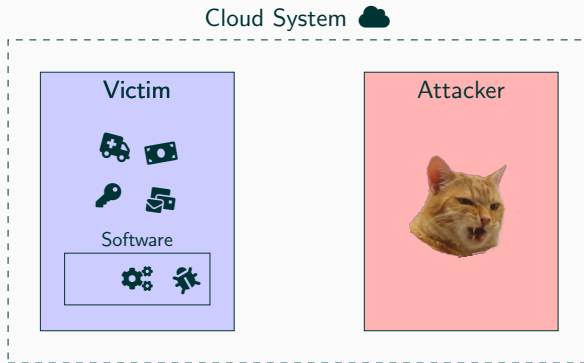




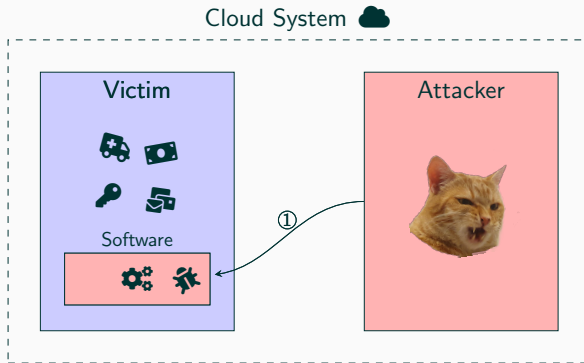
- Victim owns **secret** (data)



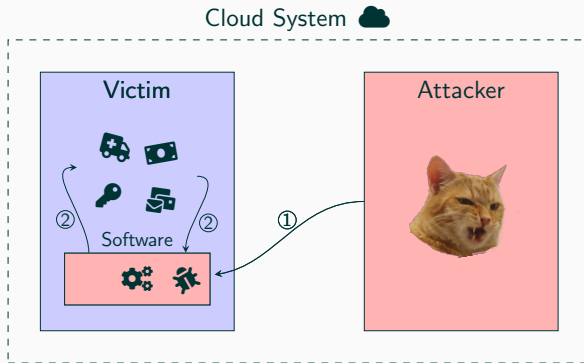
- Victim owns **secret** (data)
- **Attacker** wants to leak data



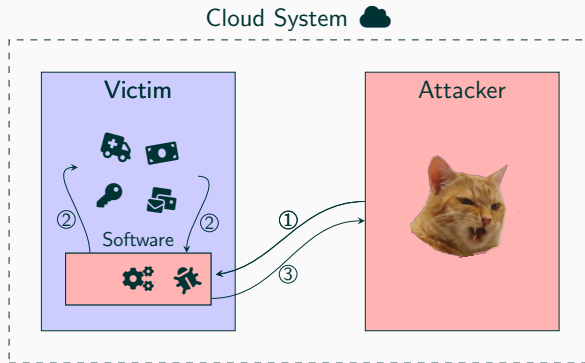
- Victim owns **secret** (data)
- **Attacker** wants to leak data
- Victim software is **buggy**



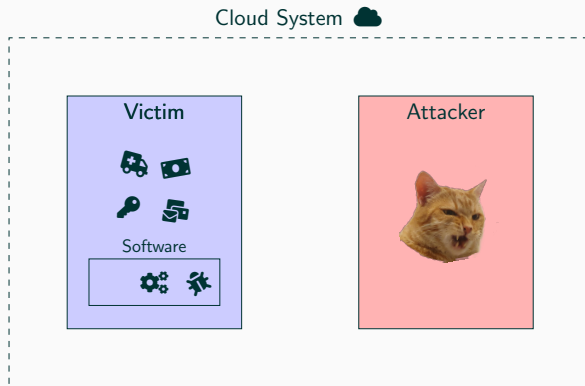
- Victim owns **secret** (data)
- **Attacker** wants to leak data
- Victim software is **buggy**
- Attacker **exploits** bug,



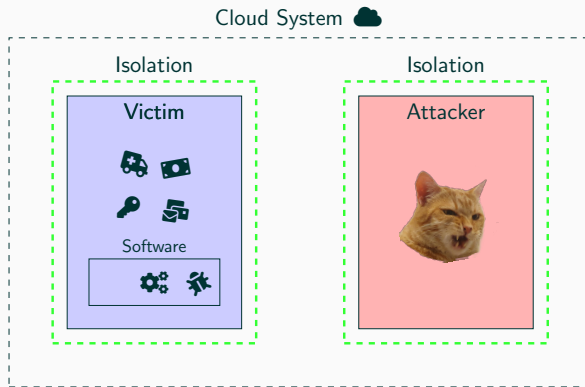
- Victim owns **secret** (data)
- **Attacker** wants to leak data
- Victim software is **buggy**
- Attacker **exploits** bug,
- reads **secrets**,



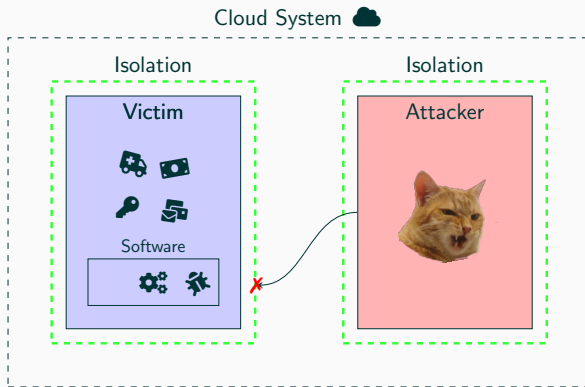
- Victim owns **secret** (data)
- **Attacker** wants to leak data
- Victim software is **buggy**
- Attacker **exploits** bug,
- reads **secrets**,
- and **extracts** them.



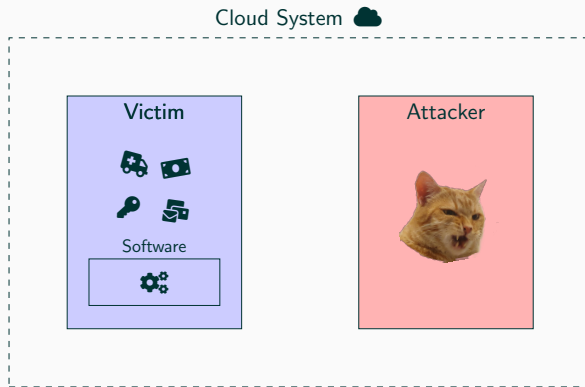
- Victim owns **secret** (data)
- Attacker wants to leak data



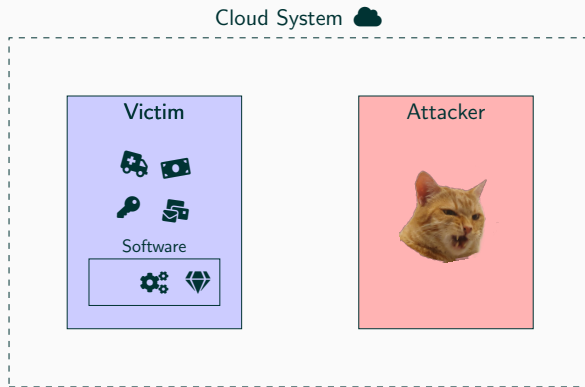
- Victim owns **secret** (data)
- Attacker wants to leak data
- Victim Software is **buggy**
- but **isolation** is enforced.



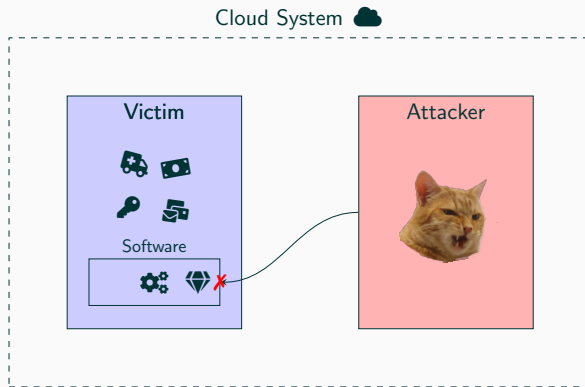
- Victim owns **secret** (data)
- Attacker wants to leak data
- Victim Software is **buggy**
- but **isolation** is enforced.
- Exploitation **fails!**



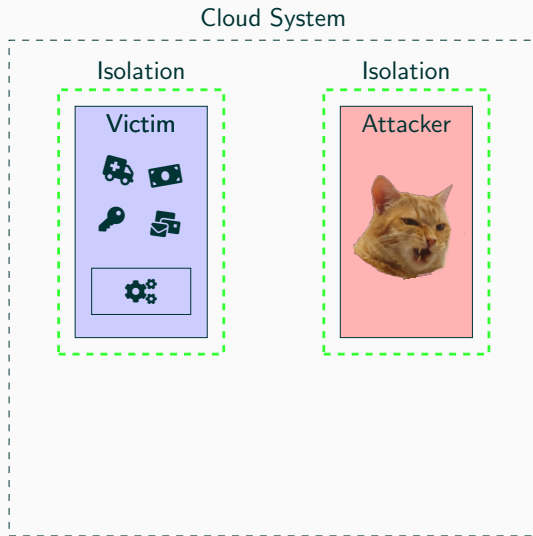
- Victim owns **secret** (data)
- Attacker wants to leak data



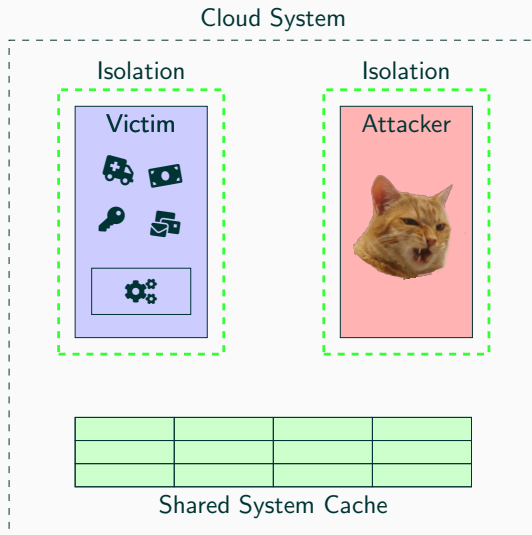
- Victim owns **secret** (data)
- Attacker wants to leak data
- Victim Software is **not** buggy



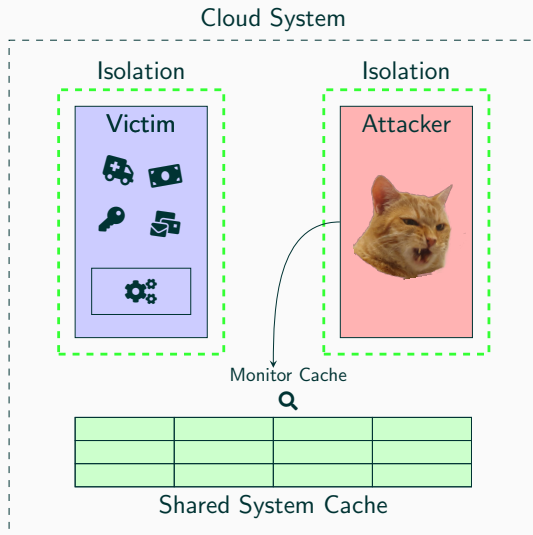
- Victim owns **secret** (data)
- Attacker wants to leak data
- Victim Software is **not** buggy
- Exploitation **fails!**



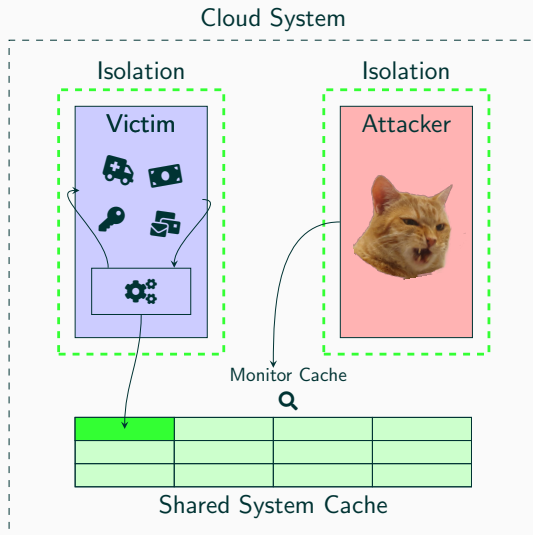
- Victim Software is **bug-free**
- **Isolation** is implemented



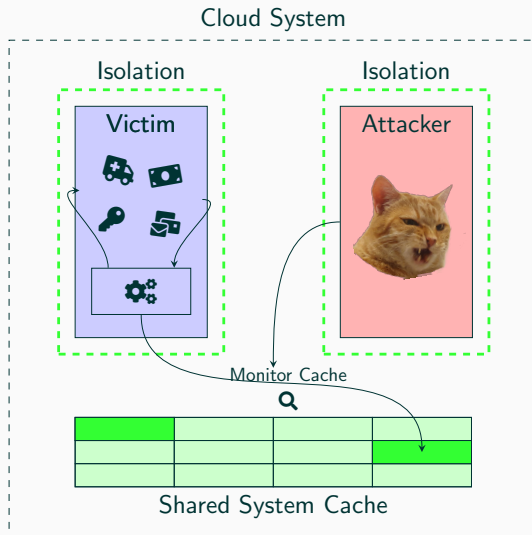
- Victim Software is **bug-free**
- **Isolation** is implemented
- System cache is **shared**



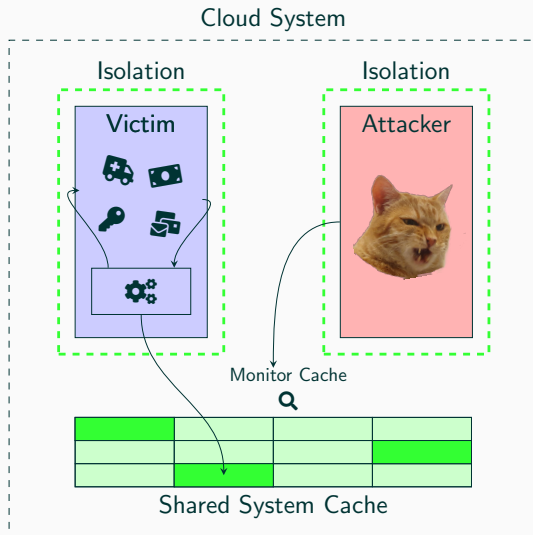
- Victim Software is **bug-free**
- **Isolation** is implemented
- System cache is **shared**
- Attacker **monitors** cache



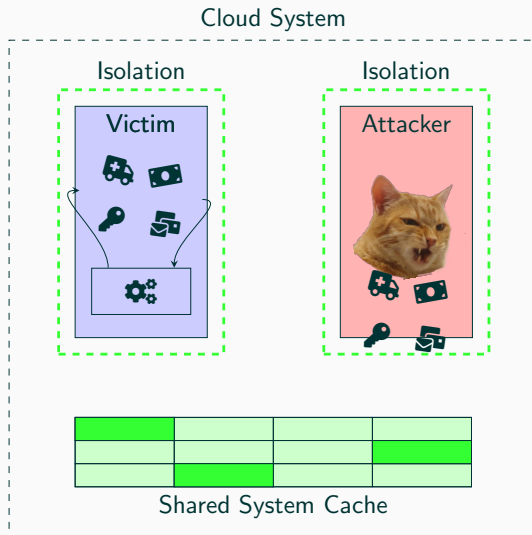
- Victim Software is **bug-free**
- **Isolation** is implemented
- System cache is **shared**
- Attacker **monitors** cache
- Victim performs **accesses**
- dependent on **secrets**



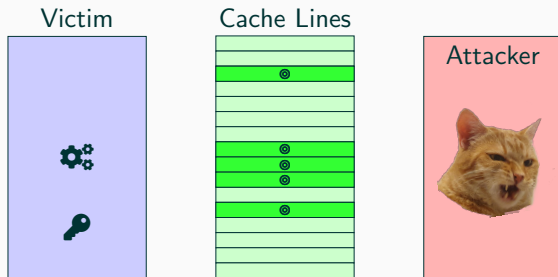
- Victim Software is **bug-free**
- **Isolation** is implemented
- System cache is **shared**
- Attacker **monitors** cache
- Victim performs **accesses**
- dependent on **secrets**



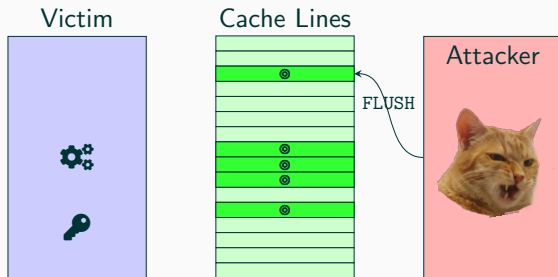
- Victim Software is **bug-free**
- **Isolation** is implemented
- System cache is **shared**
- Attacker **monitors** cache
- Victim performs **accesses**
- dependent on **secrets**
- Pattern **leaks information**



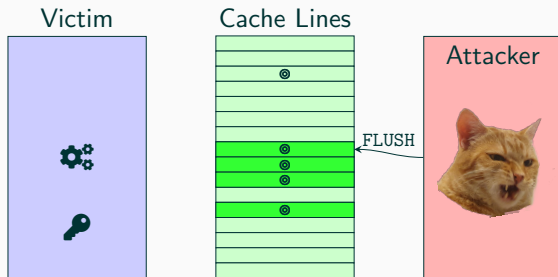
- Victim Software is **bug-free**
- **Isolation** is implemented
- System cache is **shared**
- Attacker **monitors** cache
- Victim performs **accesses**
- dependent on **secrets**
- Pattern **leaks information**
- Attacker **learns secrets**



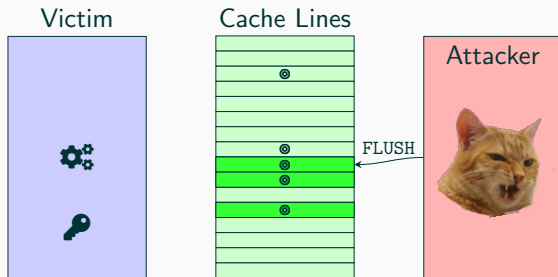
- Shared Data (eg. shared library)



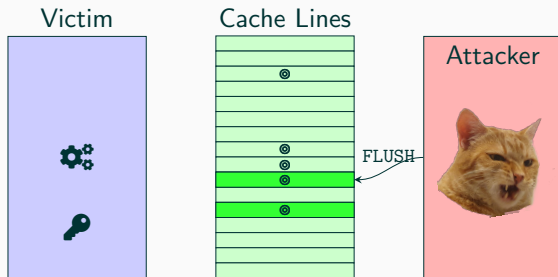
- Shared Data (eg. shared library)
- Attacker evicts cache lines



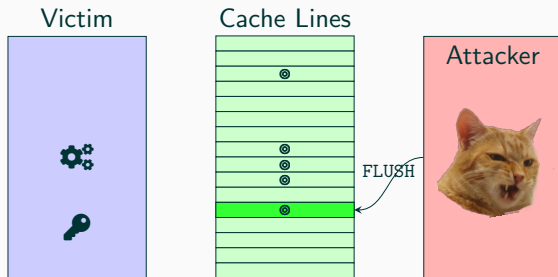
- Shared Data (eg. shared library)
- Attacker evicts cache lines



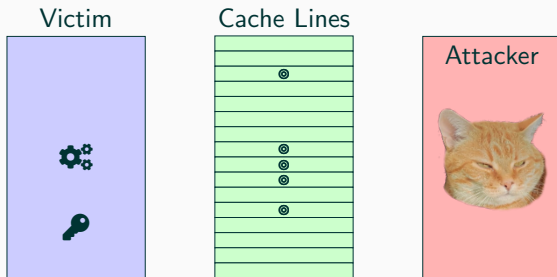
- Shared Data (eg. shared library)
- Attacker evicts cache lines



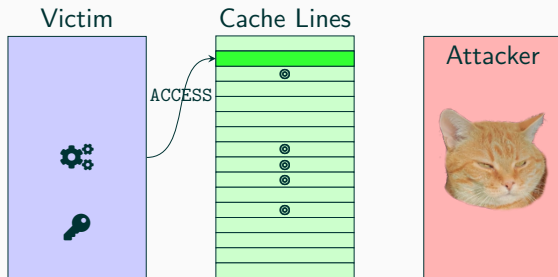
- Shared Data (eg. shared library)
- Attacker evicts cache lines



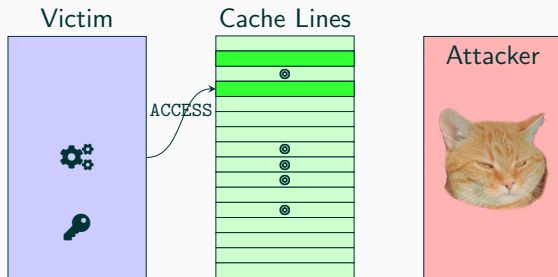
- Shared Data (eg. shared library)
- Attacker evicts cache lines



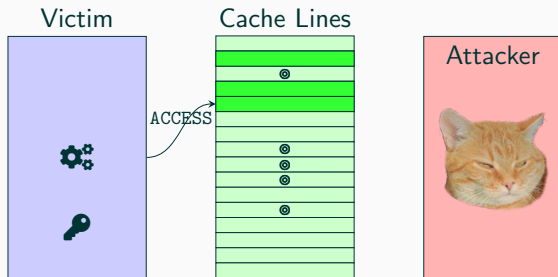
- Shared Data (eg. shared library)
- Attacker evicts cache lines



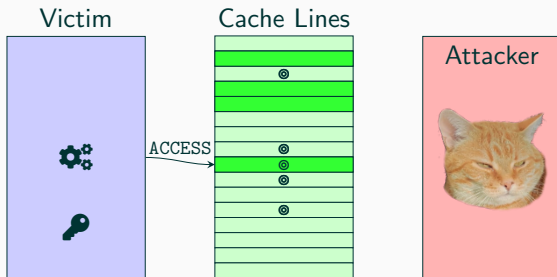
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines



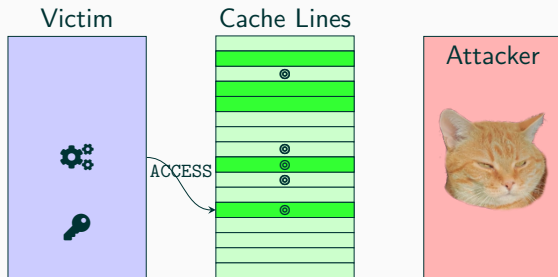
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines



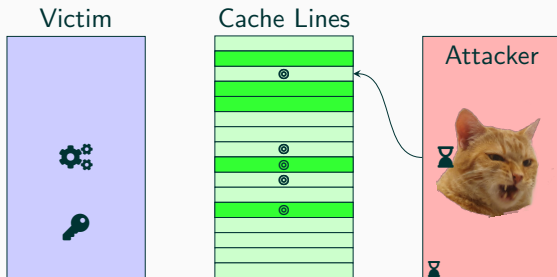
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines



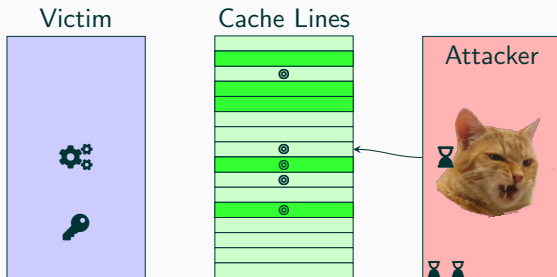
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines



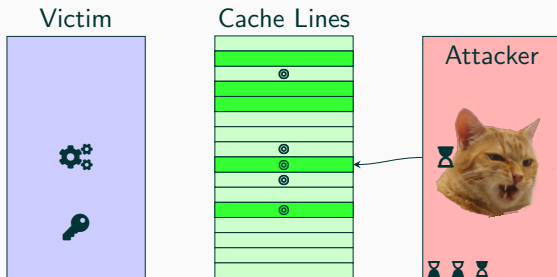
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines



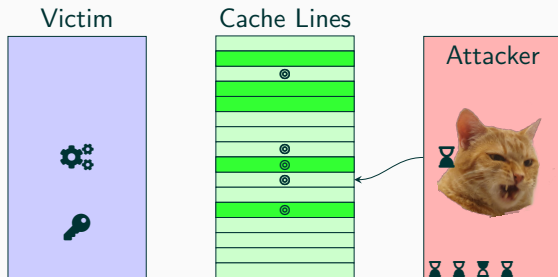
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines
- Attacker measures access latency



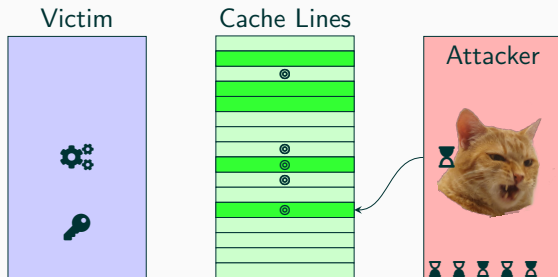
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines
- Attacker measures access latency



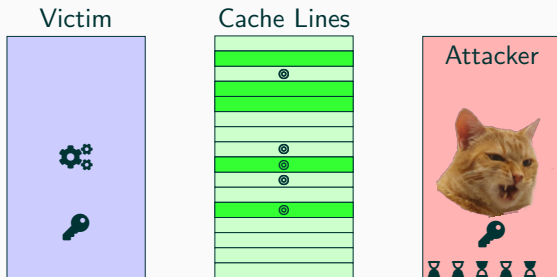
- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines
- Attacker measures access latency



- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines
- Attacker measures access latency



- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines
- Attacker measures access latency



- Shared Data (eg. shared library)
- Attacker evicts cache lines
- Victim accesses cache lines
- Attacker measures access latency
- Deduces secret from measurements

We want to detect such attacks!

General Idea:

General Idea:

- 🎯 *Monitor* vulnerable cache lines

General Idea:

- 🎯 *Monitor* vulnerable cache lines
- 🔍 *Detect* flush operations

General Idea:

- 🎯 *Monitor* vulnerable cache lines
- 🔍 *Detect* flush operations
- ⚠️ *Notify* victim

General Idea:

- 🎯 *Monitor* vulnerable cache lines
- 🔍 *Detect* flush operations
- ⚠️ *Notify* victim

Challenges

Reliably detecting CLFLUSH operations on target cache lines

General Idea:

- 🎯 *Monitor* vulnerable cache lines
- 🔍 *Detect* flush operations
- ⚠️ *Notify* victim

Challenges

Reliably detecting CLFLUSH operations on target cache lines
Without causing *too many* false positives

So far no easy way of detecting CLFLUSH

So far no easy way of detecting CLFLUSH

Recently, AMD and Intel introduced MWAITX and UMWAIT

The following additional events cause the processor to exit the implementation-dependent optimized state: a store to the address range armed by the UMONITOR instruction, an NMI or SMI, a debug exception, a machine check exception, the BINIT# signal, the INIT# signal, and the RESET# signal. Other implementation-dependent events may also cause the processor to exit the implementation-dependent optimized state.

Intel Software Developer's Manual - Description of UMWAIT

So far no easy way of detecting CLFLUSH

Recently, AMD and Intel introduced MWAITX and UMWAIT

The following additional events cause the processor to exit the implementation-dependent optimized state: a store to the address range armed by the UMONITOR instruction, an NMI or SMI, a debug exception, a machine check exception, the BINIT# signal, the INIT# signal, and the RESET# signal. Other implementation-dependent events may also cause the processor to exit the implementation-dependent optimized state.

Intel Software Developer's Manual - Description of UMWAIT

Research Question

What other events cause wakeups of UMWAIT and MWAITX?

So far no easy way of detecting CLFLUSH

Recently, AMD and Intel introduced MWAITX and UMWAIT

The following additional events cause the processor to exit the implementation-dependent optimized state: a store to the address range armed by the UMONITOR instruction, an NMI or SMI, a debug exception, a machine check exception, the BINIT# signal, the INIT# signal, and the RESET# signal. Other implementation-dependent events may also cause the processor to exit the implementation-dependent optimized state.

Intel Software Developer's Manual - Description of UMWAIT

Research Question

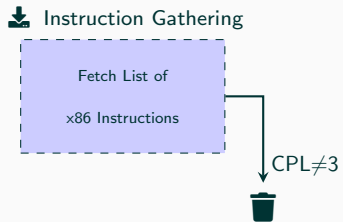
What other events cause wakeups of UMWAIT and MWAITX?

Do CLFLUSH instructions cause wakeups?

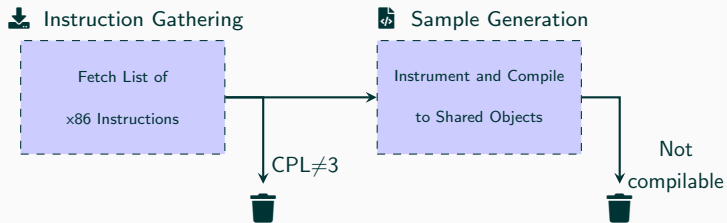
The WaitWatcher Framework

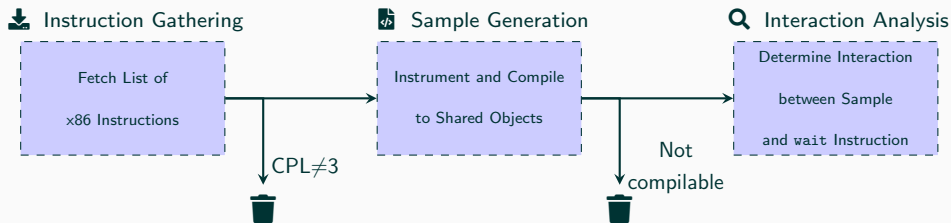
Instruction Gathering

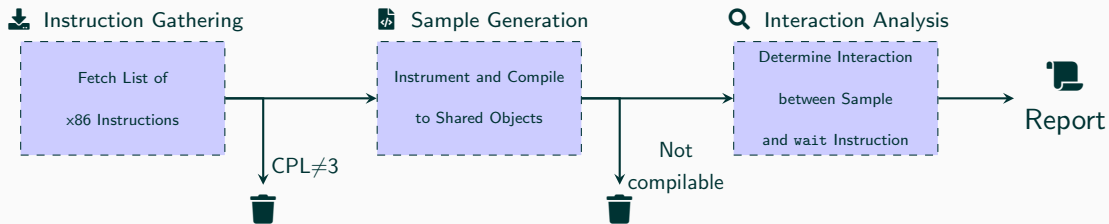
Fetch List of
x86 Instructions





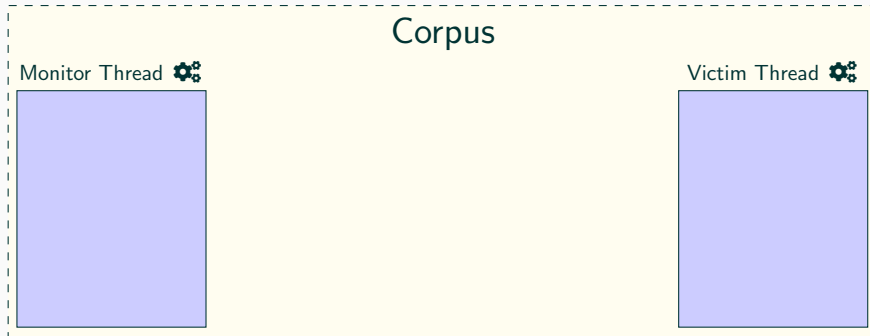


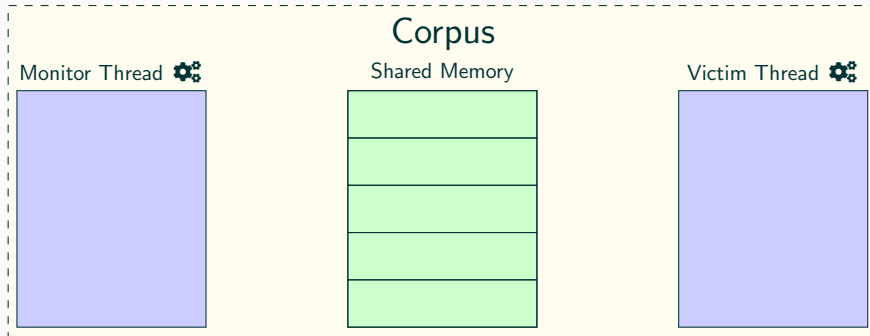


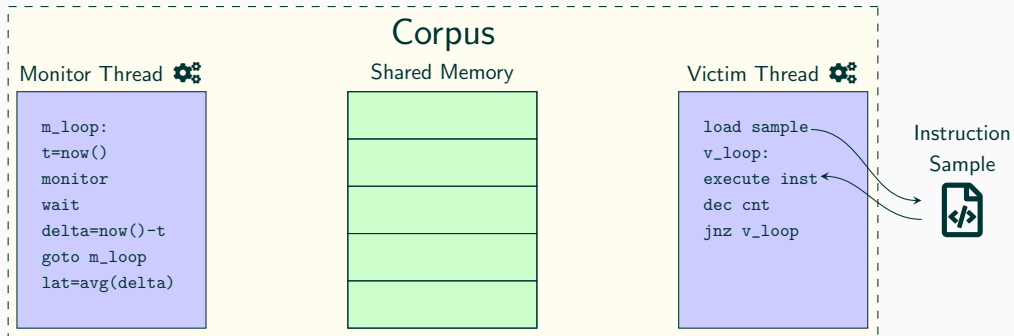


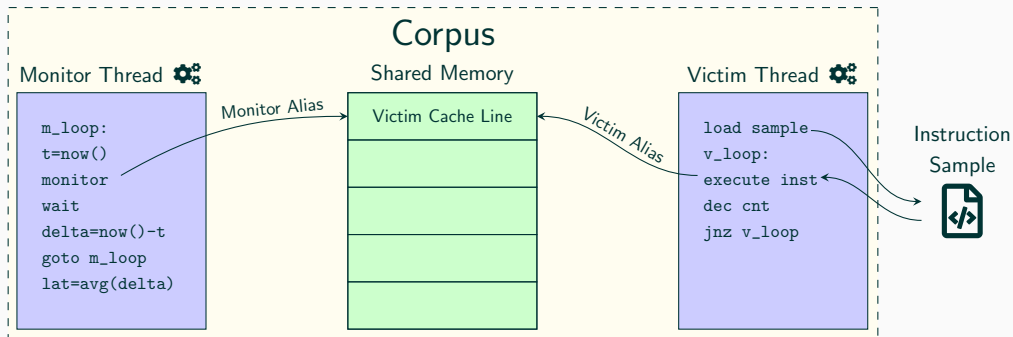


Corpus









CPU Generation	Analyzed Instructions	Data Mod.		No Data Mod.	
					
 Zen 2	2996	385	14	4	2593
 Zen 3	3005	399	0	4	2602
 Zen 4c	12 880	655	6	4	12 215
 Alder Lake	3044	389	12	3	2640
 Arrow Lake	3124	418	12	0	2692
 SPR	15 039	647	18/0 [†]	5	14 369
 EMR	15 040	647	18/0 [†]	5	14 370

 Server CPU
  Desktop CPU
 [†] Arbitrary Thread/Sibling Thread

CPU Generation	Analyzed Instructions	Data Mod.		No Data Mod.	
Zen 2	2996	385	14	4	2593
Zen 3	3005	399	0	4	2602
Zen 4c	12 880	655	6	4	12 215
Alder Lake	3044	389	12	3	2640
Arrow Lake	3124	418	12	0	2692
SPR	15 039	647	18/0 [†]	5	14 369
EMR	15 040	647	18/0 [†]	5	14 370

Server CPU
 Desktop CPU
 † Arbitrary Thread/Sibling Thread

Findings

CLFLUSH causes wakeups!

CPU Generation	Analyzed Instructions	Data Mod.		No Data Mod.	
Zen 2	2996	385	14	4	2593
Zen 3	3005	399	0	4	2602
Zen 4c	12 880	655	6	4	12 215
Alder Lake	3044	389	12	3	2640
Arrow Lake	3124	418	12	0	2692
SPR	15 039	647	18/0 [†]	5	14 369
EMR	15 040	647	18/0 [†]	5	14 370

Server CPU
 Desktop CPU
 † Arbitrary Thread/Sibling Thread

Findings

CLFLUSH causes wakeups!

Some instructions bypass the wakeup logic

CPU Generation	Analyzed Instructions	Data Mod.		No Data Mod.	
Zen 2	2996	385	14	4	2593
Zen 3	3005	399	0	4	2602
Zen 4c	12 880	655	6	4	12 215
Alder Lake	3044	389	12	3	2640
Arrow Lake	3124	418	12	0	2692
SPR	15 039	647	18/0 [†]	5	14 369
EMR	15 040	647	18/0 [†]	5	14 370

Server CPU
 Desktop CPU
 † Arbitrary Thread/Sibling Thread

Findings

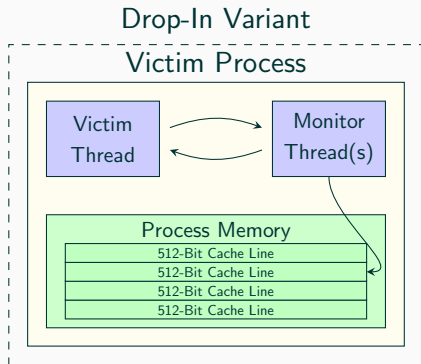
CLFLUSH causes wakeups!

Some instructions bypass the wakeup logic

AMD systems only wake up for sibling threads

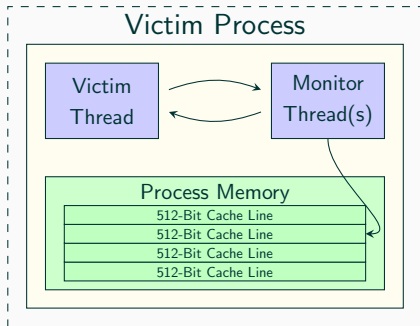
WaitGuard and Evaluation

Idea: detect CLFLUSH using UMWAIT

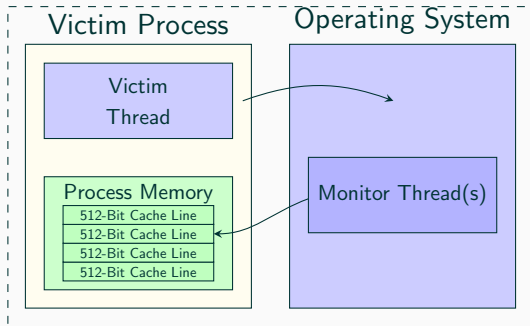


Idea: detect CLFLUSH using UMWAIT

Drop-In Variant

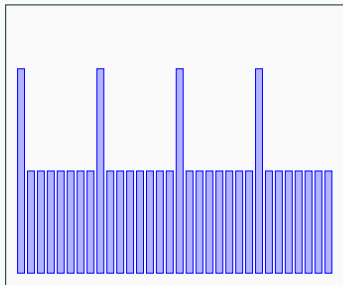


Delegated Variant



Implement three different Flush+Reload Attackers

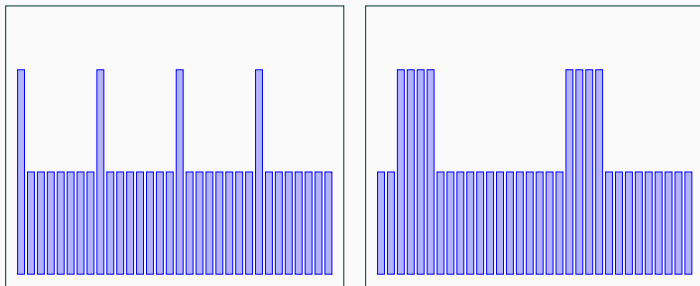
$\mathcal{V}_1$ Attack T-Table cache lines one after the other



Implement three different Flush+Reload Attackers

$\mathcal{V}_1$ Attack T-Table cache lines one after the other

$\mathcal{V}_2$ Flush + Reload on all T-Table cache lines

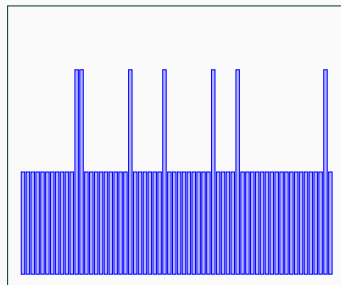
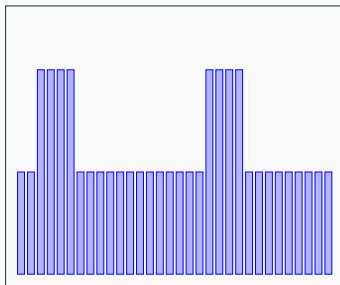
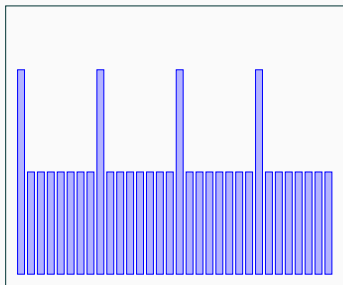


Implement three different Flush+Reload Attackers

$\mathcal{V}_1$ Attack T-Table cache lines one after the other

$\mathcal{V}_2$ Flush + Reload on all T-Table cache lines

$\mathcal{V}_3$ 0.2% chance of attack for each iteration (stealthy)



Implement Proof-Of-Concept and attack victims $\mathcal{V}_1$ to $\mathcal{V}_3$

Implement Proof-Of-Concept and attack victims $\mathcal{V}_1$ to $\mathcal{V}_3$

❏ Test different window sizes to filter false positives

Implement Proof-Of-Concept and attack victims $\mathcal{V}_1$ to $\mathcal{V}_3$

- Test different window sizes to filter false positives

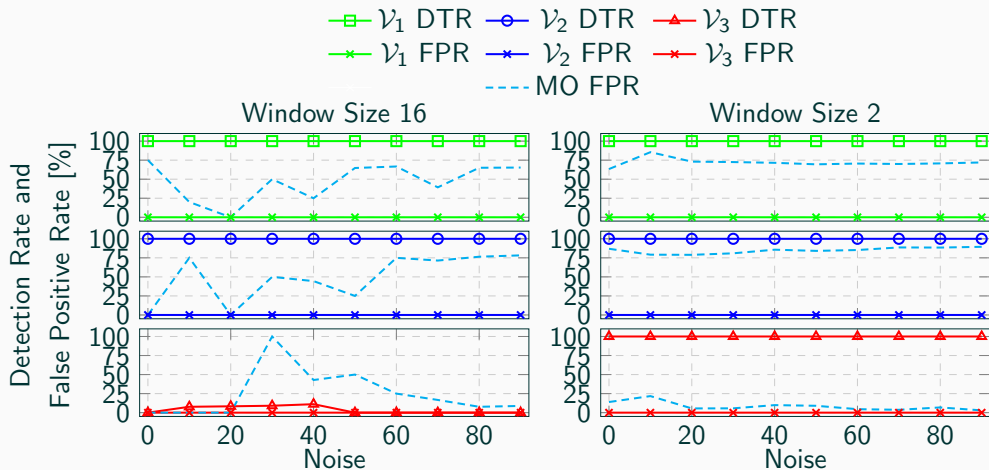
- Apply system noise using `stress-ng`

Implement Proof-Of-Concept and attack victims $\mathcal{V}_1$ to $\mathcal{V}_3$

 Test different window sizes to filter false positives

 Apply system noise using stress-ng

 Compare to Measure-Only approach



Implement Proof-Of-Concept and attack victims $\mathcal{V}_1$ to $\mathcal{V}_3$

🔍 Test different window sizes to filter false positives

🔊 Apply system noise using stress-ng

📊 Compare to Measure-Only approach

Results

We reach 100% detection rate with $< 0.1\%$ false positives!

Downside of UMWAIT: Only one cache line per logical thread

Downside of UMWAIT: Only one cache line per logical thread

- 🕒 Schedule monitoring for multiple cache lines

Downside of UMWAIT: Only one cache line per logical thread

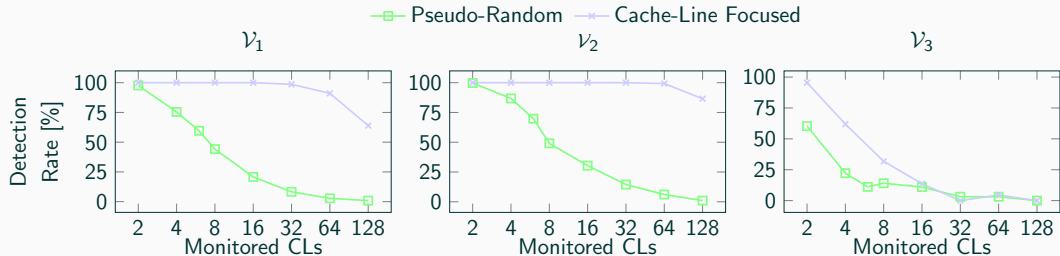
- 🕒 Schedule monitoring for multiple cache lines
- 🔀 Implement pseudo-random scheduling

Downside of UMWAIT: Only one cache line per logical thread

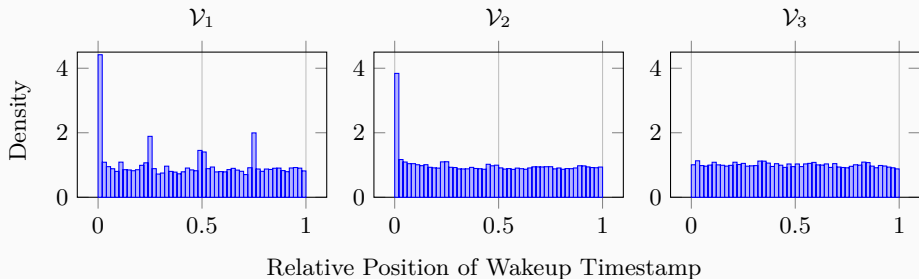
- 🕒 Schedule monitoring for multiple cache lines
- 🔗 Implement pseudo-random scheduling
- 🔍 and cache-line focused scheduling

Downside of UMWAIT: Only one cache line per logical thread

- 🕒 Schedule monitoring for multiple cache lines
- ⚡ Implement pseudo-random scheduling
- 🔍 and cache-line focused scheduling



When does WaitGuard detect an attack?



Results

We detect Flush+Reload attacks early in the attack process!

Conclusion

👁 WAIT instructions wake up on non-modifying instructions

- 👁 WAIT instructions wake up on non-modifying instructions
- 🚫 WAIT instructions can miss data-modifying instructions

- 👁 WAIT instructions wake up on non-modifying instructions
- 🚫 WAIT instructions can miss data-modifying instructions
- ? Weird behavior on AMD systems

- 👁 WAIT instructions wake up on non-modifying instructions
- 🚫 WAIT instructions can miss data-modifying instructions
- ❓ Weird behavior on AMD systems
- 🔍 WAIT instructions detect CLFLUSH (on some systems)

- 👁 WAIT instructions wake up on non-modifying instructions
- 🚫 WAIT instructions can miss data-modifying instructions
- ❓ Weird behavior on AMD systems
- 🔍 WAIT instructions detect CLFLUSH (on some systems)
- 📊 WaitGuard achieves a high detection rate with virtually no false positives

WaitWatcher & WaitGuard

Detecting Cache Side Channels through Spurious Wakeups

Lukas Lamster

September 22, 2025

ISEC – Graz University of Technology